Multi-Objective Optimization for Image Classification

MOML Project Report

Velidanda Krishna Sai (IMT2023094)
Chaitya Shah (IMT2023055)

Course: Multi-Objective Machine Learning
Instructor: Prof. Aswin Kannan

April 28, 2026

1 What we are trying to do

The idea of this project is to pick a small CNN image classifier and try to make optimise three objectives:

1. **Be accurate** – get the highest top-1 accuracy on the test set (we want this to be *big*).
2. **Be fast** – take the shortest possible time, in milliseconds, to classify one image on the CPU (we want this to be *small*).
3. **Be small** – have the fewest trainable parameters possible (we want this to be *small* too).

These three objectives conflict each other. If you make the model bigger to get more accuracy, it usually becomes slower and heavier. So there is no single "best" model – there are many models that are good in different ways. The set of models where you can't improve one objective without hurting another is called the **Pareto front**, and finding it (and understanding it) is the whole point of multi-objective optimization (MOO).

We chose the **Image Classification** application from the project description and used **Fashion-MNIST** as the dataset (28×28 grayscale images, 10 classes, 60k train / 10k test). We compared two MOO frameworks head-to-head on the same search space and the same trial budget:

- **pymoo NSGA-II** – an evolutionary algorithm.
- **BoTorch qNEHVI** – a Bayesian optimization method using Gaussian Processes.

Both got **80 trials** each, with a wall-clock budget of about 1.5 hours per run (the spec said under 2 hours).

2 The setup

2.1 The CNN search space

Each "trial" picks one configuration from this search space, builds a CNN with those settings, trains it, and measures the three objectives. The space is the same for both frameworks (this is important for a fair comparison):

Variable	Type	Values
`arch_type`	fixed	"plain"
`num_conv_layers`	choice	1, 2, or 4
`num_channels`	choice	8, 16, or 32
`num_fc_units`	fixed	128
`learning_rate`	cont. (log)	$[10^{-5}, 10^{-2}]$
`batch_size`	choice	16 or 64
`num_epochs`	integer	5 to 10
`dropout_rate`	continuous	0.0 to 0.5
`optimizer_type`	choice	SGD or Adam
`input_resolution`	choice	16 or 28

Table 1: Search space (identical across both frameworks).

2.2 How we measure each objective

For each trial we do this loop:

1. Build a CNN using the sampled config.
2. Train it for the chosen number of epochs on a fixed random subset of 8000 images (for speed).
3. **Accuracy:** run the trained model on the full test set, compute top-1.
4. **Inference time:** move the model to **CPU**, do 50 warmup runs and 500 timed runs with `batch_size = 1` using `torch.no_grad()` and `time.perf_counter()`. Report the average milliseconds per sample. (CPU + batch=1 is what the spec asks for.)
5. **Param count:** just `sum(p.numel() for p in model.parameters() if p.requires_grad)`.

The trial seed is always `base_seed + trial_index`, so everything is reproducible.

2.3 The two frameworks

pymoo NSGA-II. Classic evolutionary MOO. We used population size 20 and 4 generations, which gives $20 \times 4 = 80$ evaluations. NSGA-II uses non-dominated sorting and crowding distance to keep a diverse population.

BoTorch qNEHVI. Bayesian MOO. It fits a Gaussian Process surrogate over the objective space and uses the qNEHVI acquisition function *(q-Noisy Expected Hypervolume Improvement)* to pick the next configuration that is expected to improve the hypervolume the most. We used 16 random initial points and then 64 BO suggestions = 80 total.

Both frameworks got the same total budget, the same dataset, the same training subset size, and the same seed (42). The only thing different is *how* they pick the next configuration to try.

3 Quantitative results

3.1 Headline numbers

Metric	pymoo	BoTorch
Total trials	80	80
Pareto points found	18	21
Wall-clock (s)	4727	6165
Hypervolume	$\mathbf{8.77\times10^9}$	8.73×10^9
Spacing (lower = better)	**0.070**	0.170
Best accuracy	**0.8783**	0.8739
Min inference (ms)	0.257	**0.217**
Min param count	2538	2538

Table 2: Side-by-side summary on Fashion-MNIST. Reference point for hypervolume: $[0, 1000\text{ms}, 10\text{M params}]$.

A few quick observations:

- Both frameworks converged to nearly identical hypervolume values, indicating that each effectively performed meaningful multi-objective optimization rather than behaving like random search. Notably, pymoo achieved a marginally higher hypervolume, suggesting a slight performance advantage.
- pymoo's spacing is way better (0.070 vs 0.170). BoTorch's Pareto points cluster more, while pymoo's are spread more evenly along the front. This makes sense: NSGA-II actively encourages diversity via crowding distance, while BoTorch tries to maximize hypervolume gain and doesn't explicitly care about spread.
- BoTorch found *more* non-dominated points (21 vs 18) but they are bunched up.
- BoTorch was slower in wall-clock (25 min more). That's the cost of refitting the GP and optimizing the acquisition function every step.
- Both frameworks happened to find the same smallest model (2538 params, a tiny 1-conv-layer net at 16×16 input).

3.2 Joint Pareto front

The really interesting comparison: when we put both frameworks' Pareto points together and recompute the Pareto front, who survives?

- Joint Pareto size: **25 points**.
- Surviving from BoTorch: **19 of 21**.
- Surviving from pymoo: only **6 of 18**.

So even though pymoo's hypervolume is slightly higher (probably because it pushed further at the extremes), most of pymoo's "Pareto" points get *dominated* when you let BoTorch's points compete. BoTorch found a denser, generally better-quality front in the middle of the objective space, while pymoo did better at the extremes.

Generational distance (GD) of each front to the joint front is roughly equal: **0.067 (BoTorch)** vs **0.067 (pymoo)** – so on average the points are equally close to the true joint front. The difference is really about *where* on the front each one is dense.

3.3 Plots

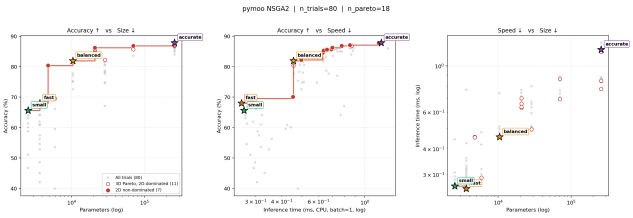

Figure 1: pymoo NSGA-II: 2D pairwise scatter (all trials grey, Pareto red).

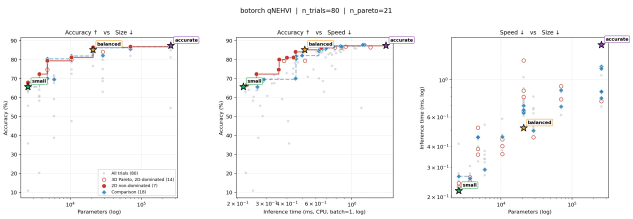

Figure 2: BoTorch qNEHVI: 2D pairwise scatter (with pymoo overlay).

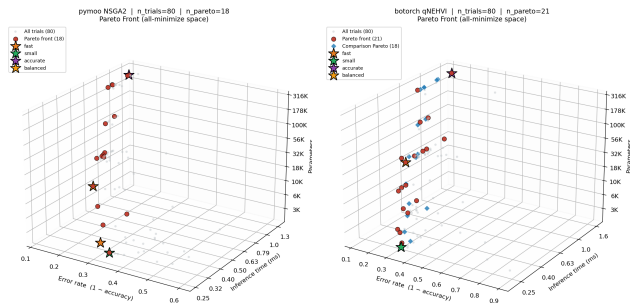

Figure 3: 3D Pareto fronts. Left: pymoo. Right: BoTorch.

4 Qualitative analysis

4.1 The trade-offs we actually saw

Looking at the Pareto points (Section 5), here are the trade-offs that stand out:

Accuracy costs a lot of parameters, but not that much speed. Going from the smallest Pareto model

(∼2.5k params, 65.6% accuracy) to the most accurate one (∼258k params, 87.8% accuracy) is a **100× increase in model size** for a **22 percentage point gain in accuracy**. But inference time only goes from 0.26 ms to 1.20 ms – only **4.5×**. So if you have a strict size budget, you pay a lot. If you only care about speed, the penalty is much milder.

The middle of the front is where the value is. The "balanced" point we picked (around 82% accuracy, 0.45 ms, 10k params) gets you almost all the accuracy of the best model with about **1/25th the parameters** and about **2.6× faster** inference. For most real-world use this is the much smarter choice.

4.2 One non-trivial trade-off

We expected "more conv layers → slower inference" to be straight-line. It isn't.

In our results, a 4-layer model with 8 channels at 16×16 input runs in **0.65 ms with 20.8k params and 86.2% accuracy**, while a 4-layer model with 32 channels at 28×28 input runs in **1.20 ms with 258k params and 87.8% accuracy**. Same depth, but the second is **12× more parameters and only 1.6 percentage points more accurate**. The number of channels and input resolution dominate inference time and parameter count much more than depth does. That was not obvious to us going in – we kept thinking "deeper = slower", but really "wider + bigger input = much slower and much heavier, for not much accuracy".

4.3 Strengths and weaknesses

pymoo NSGA-II strengths:

- Spread out Pareto points evenly (much better spacing).
- Faster wall-clock (no GP fitting overhead).
- Much simpler to set up and reason about; mixed-variable mating works out of the box.
- Pushed harder at the extremes (highest accuracy point, very small model).

pymoo NSGA-II weaknesses:

- With only 80 evaluations and pop=20 / gen=4, the population doesn't have time to really converge – we only get 4 generations. Many of pymoo's Pareto points end up dominated when BoTorch's are added in.
- Doesn't use any information from past trials beyond "which are non-dominated".

BoTorch qNEHVI strengths:

- Sample-efficient – 19 of its 21 Pareto points survive in the joint front. It really does learn the objective surface.
- Found a denser front in the practically useful middle region.

BoTorch qNEHVI weaknesses:

- Slower wall-clock per trial because GP refitting and acquisition optimization aren't free.
- Spacing is much worse – it groups points together rather than spreading them out, which is annoying if you actually want to choose between distinct trade-offs.
- More fiddly to set up (objective normalization, MC samples, restarts, etc.).

Practical recommendation: for this kind of NAS-style problem with a moderate budget (∼80 trials), BoTorch gives slightly better-quality non-dominated points but pymoo gives a more decision-friendly front. If you care about "give me the best 4 distinct trade-offs to choose between", pymoo wins on user experience. If we care about "find the absolute best non-dominated solutions you can in 80 evaluations", BoTorch performs marginally better.

5 Pareto points table (4 per framework)

We picked four practically useful Pareto points from each run: **fast** (lowest inference), **small** (lowest param count), **accurate** (highest accuracy), and **balanced** (closest to the ideal point in normalized objective space).

Framework	Label	Acc. (%)	Inf. (ms)	Params
pymoo	fast	68.02	0.257	3,658
pymoo	small	65.61	0.264	2,538
pymoo	accurate	87.83	1.200	258,762
pymoo	balanced	81.93	0.455	10,410
BoTorch	fast	65.69	0.217	2,538
BoTorch	small	65.69	0.217	2,538
BoTorch	accurate	87.39	1.620	258,762
BoTorch	balanced	85.27	0.514	20,826

Table 3: Four Pareto-optimal solutions per framework. For BoTorch, fast and small are the same config.

A small thing to point out: pymoo's "balanced" point gives 81.9% accuracy with only 10k params, while BoTorch's "balanced" is 85.3% accuracy with 21k params. Both are reasonable, just at different points on the trade-off curve – BoTorch traded model size for accuracy, pymoo did the opposite.

6 Bonus experiment: a bigger pymoo run (not directly comparable)

Following the initial head-to-head comparison, we conducted a second, substantially larger-scale experiment

using pymoo to evaluate the performance of NSGA-II under increased computational budget. This experiment is reported separately, as it is not directly comparable to the 80-trial study; multiple experimental factors were modified simultaneously, precluding an apples-to-apples comparison.

6.1 What's different from the head-to-head

- **Full training data per trial.** The lite runs train on an 8000-image subset; this run trains on the full 60,000-image Fashion-MNIST training set every trial. Each trial sees 7.5× more data.
- **Bigger budget.** pop_size = 40, n_gen = 5 → **200 trials** instead of 80.
- **Mixed-precision (AMP)** enabled on GPU – fp16 autocast + GradScaler. Roughly 2× faster training per trial on Ampere/Ada GPUs.
- **Wider search space** on a couple of dims: `num_fc_units` $\in \{64, 128, 256\}$ and `batch_size` $\in \{64, 128, 256\}$ (the lite runs fixed FC units to 128 and used only $\{16, 64\}$ for batch size).
- **Lighter inference timing**: 10 warmup + 100 timed CPU passes instead of 50 + 500. Variance is still well under 1 ms but the per-trial time saving adds up over 200 trials.

6.2 Results

Metric	pymoo (full)
Total trials	200
Pareto points (over all trials)	48
Pareto points (final population)	40
Wall-clock (s)	2872 (48 min, GPU)
Hypervolume	9.21×10^9
Spacing	0.0625
Best accuracy	0.9218
Min inference (ms)	0.122
Min param count	1,322

Table 4: Bonus pymoo run on full Fashion-MNIST (200 trials).

Label	Acc. (%)	Inf. (ms)	Params
fast	70.99	0.122	2,538
small	71.35	0.157	1,322
accurate	92.18	2.685	276,554
balanced	86.54	0.228	15,914

Table 5: Four Pareto picks from the bonus run.

6.3 What the bigger MOO results in

Compared to the lite pymoo run (Section 2), the front pushes out in every direction:

- Best accuracy goes from **87.83%** → **92.18%** (+4.4 pp). This is mostly the full-data effect – training on 60k images instead of 8k gives the bigger architectures room to actually learn.
- Smallest model goes from 2538 → **1322 params**. That's a tiny 1-conv 64-FC network – only available because the wider search space allows `num_fc_units = 64`.
- Fastest inference goes from 0.257 ms → **0.122 ms**, basically a 2× speedup at the speed extreme.
- Hypervolume goes from 8.77×10^9 → 9.21×10^9, and spacing improves slightly (0.070 → 0.063). With 5 generations and pop=40, NSGA-II has more time to actually evolve and spread.

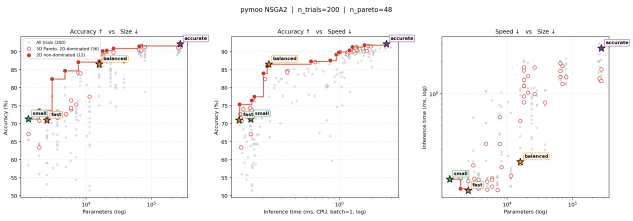

Figure 4: Bonus pymoo run: 2D pairwise scatter (200 trials, full data).

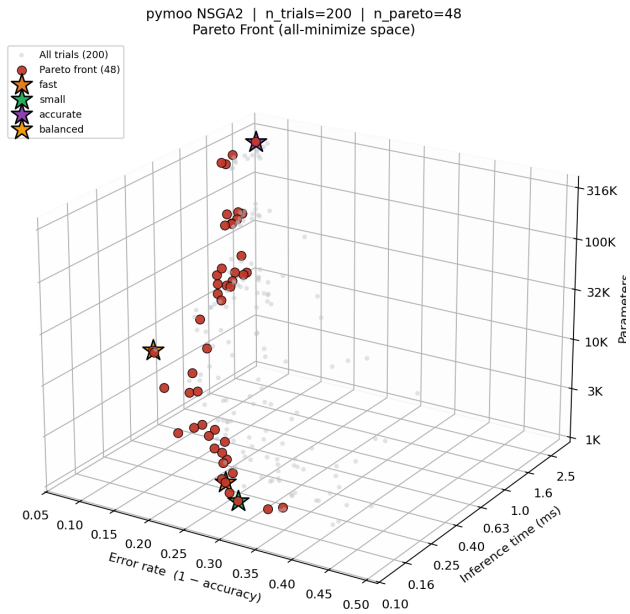

Figure 5: Bonus pymoo run: 3D pairwise scatter (200 trials, full data).

7 Appendix: one Pareto-optimal solution explained in detail

We pick the **balanced** solution from the pymoo NSGA-II run. This is the Pareto point closest to the ideal point in normalized objective space, so it is the most "well-rounded" trade-off the search found.

Trial info

- Framework: pymoo NSGA-II
- Trial number: 68 (out of 80)

- Seed: 110 (= base seed 42 + 68)

Decision variables (the architecture and training config)

- `arch_type`: plain CNN
- `num_conv_layers`: 2
- `num_channels`: 16 (per conv layer)
- `num_fc_units`: 128
- `learning_rate`: 0.00774
- `batch_size`: 16
- `num_epochs`: 8
- `dropout_rate`: 0.0049
- `optimizer_type`: SGD
- `input_resolution`: 16×16 (downsampled from 28×28)

Objective values measured

- Accuracy: **81.93%**
- Inference time (CPU, batch=1): **0.455 ms / sample**
- Param count: **10,410**

Why this point is interesting. 81.9% accuracy on Fashion-MNIST with only **10k parameters** and sub-half-millisecond inference is a really nice operating point. Compared to the most accurate model in the same run (87.8% with 258k params, 1.20 ms), this gives you about 93% of the accuracy with only **4% of the parameters** and runs **2.6× faster**. For an embedded or low-power deployment, this is the model you'd actually ship.

The configuration itself makes sense: 2 conv layers with 16 channels each at low input resolution keeps the parameter count tiny, SGD with a moderate learning rate trains stably for 8 epochs, and almost no dropout (0.005) is fine because the small training subset doesn't really overfit at this scale.

8 Reproducibility

Code: https://github.com/melohub-xbit/MOML_Project. To reproduce:

```
# pymoo
python src/moo_pymoo.py --dataset fashion_mnist \
  --pop-size 20 --n-gen 4 \
  --train-subset-size 8000 --seed 42

# BoTorch
python src/moo_botorch.py --dataset fashion_mnist \
  --total-trials 80 --n-init 16 \
  --train-subset-size 8000 --seed 42

# analysis
python src/analyze_study.py --study-dir <path> \
  --compare <other-pareto.csv>
```

Both frameworks share the trial loop (`train_eval.py`), the loaders (`data_loader.py`), and the model factory (`models.py`). The search space, trials.csv schema, and Pareto extraction in `pareto_analysis.py` are framework-agnostic, so swapping in another MOO algorithm later is a small change.